Лабораторная работа №2
Основы обработки цифровых изображений
(Вариант 8)

Лепеев Вадим

22 октября 2026 г.

1 Цель работы

Изучить базовые алгоритмы обработки цифровых изображений, принципы пространственной фильтрации и сегментации (бинаризации). Реализовать приложение, позволяющее применять высокочастотный фильтр (увеличение резкости) и два метода глобальной пороговой обработки (с фиксированным порогом и алгоритм Оцу). Сформировать базу тестовых изображений, покрывающую краевые случаи освещённости и расфокусировки, для проведения сравнительного анализа алгоритмов.

2 Математика и алгоритмы

2.1 Высокочастотный фильтр (Увеличение резкости)

Повышение резкости достигается за счёт усиления высокочастотных компонент изображения (перепадов яркости, контуров). Математически это реализуется через операцию дискретной двумерной свёртки исходного изображения $f(x, y)$ с ядром H:

$$g(x, y) = \sum_{i=-1}^{1} \sum_{j=-1}^{1} f(x + i, y + j) \cdot H(i, j)$$

В данной работе используется ядро на основе оператора Лапласа, которое вычисляет разность между центральным пикселем и его соседями:

$$H = \begin{pmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{pmatrix}$$

2.2 Глобальная пороговая обработка (Бинаризация)

Бинаризация переводит полутоновое изображение в чёрно-белое.

1. Базовый метод (фиксированный порог): Каждый пиксель $f(x, y)$ независимо сравнивается с константой T (в данной работе $T = 127$).

$$g(x, y) = \begin{cases} 255, & \text{если } f(x, y) > T \\ 0, & \text{если } f(x, y) \leq T \end{cases}$$

2. Метод Оцу (Otsu): Алгоритм вычисляет оптимальный порог T автоматически. Вместо попиксельного анализа он анализирует статистику всего изображения (гистограмму). Порог выбирается так, чтобы максимизировать межклассовую дисперсию между «чёрными» и «белыми» пикселями:

$$\sigma_B^2(T) = \omega_0(T)\omega_1(T)[\mu_0(T) - \mu_1(T)]^2 \to \max$$

3 Параметры окружения и используемый стек

- Язык программирования: Python 3
- Библиотека компьютерного зрения: OpenCV (`cv2`)
- Векторные вычисления и матрицы: NumPy
- Графический интерфейс: CustomTkinter, Pillow (PIL)
- Запуск: `python main.py`

4 Формирование тестовой базы и интерфейс

Для тестирования уязвимостей алгоритмов была сгенерирована база из 8 изображений, моделирующих реальные проблемы:

1. Размытый текст, размытая геометрия, имитация расфокусированного медицинского снимка (для ВЧ-фильтра).
2. Нормальный текст, сильно недосвеченный текст (тень), пересвеченный текст (блик), штрихкод в условиях низкой освещенности, светлые объекты на тёмном фоне (для бинаризации).

Приложение предоставляет удобный интерфейс для загрузки этих изображений и применения методов «на лету». Перерасчёт матриц выполняется библиотекой OpenCV.

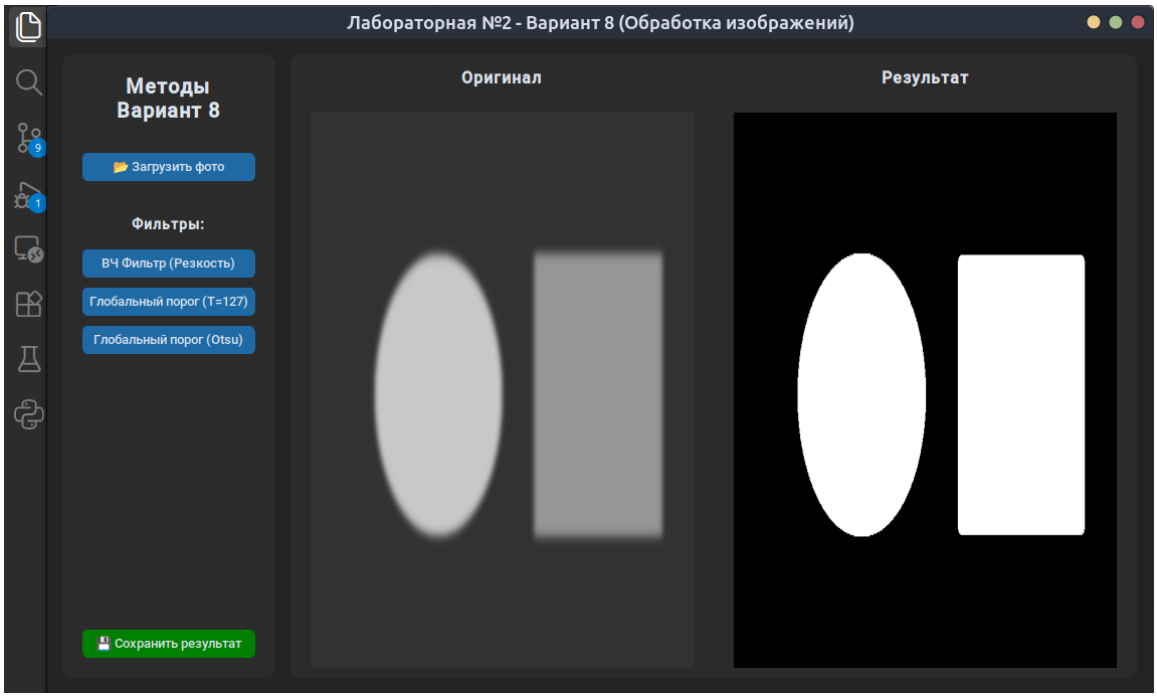


Рис. 1: Главное окно программы с загруженным тестовым изображением

5 Анализ работы алгоритмов

5.1 Пространственная фильтрация (Резкость)

При тестировании на сильно размытом тексте ВЧ-фильтр успешно восстановил границы символов. За счёт матричного умножения с ядром Лапласа, градиент яркости на краях был математически усилен, что сделало текст читаемым.

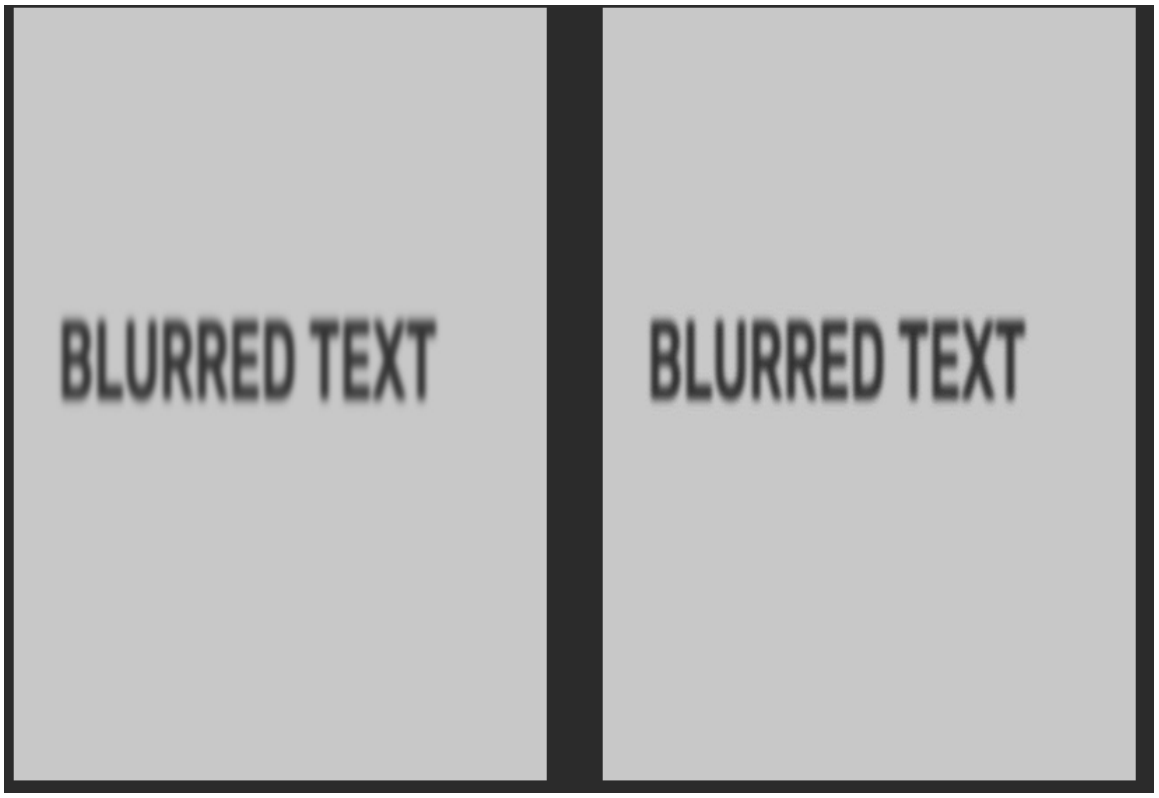


Рис. 2: Восстановление резкости размытого текста высокочастотным фильтром

5.2 Сравнение методов бинаризации (Краевые случаи)

При тестировании на нормальном изображении оба метода отработали корректно. Однако главная разница проявилась на краевых случаях освещённости.

Случай 1: Недосвеченное изображение (тень). Яркость фона составила 80, яркость текста 10. Базовый метод с порогом 127 закрасил всё изображение в чёрный цвет, так как ни один пиксель не преодолел порог. Алгоритм Оцу динамически вычислил порог ≈ 45 и идеально выделил текст.



Рис. 3: Сбой базового метода ($T = 127$) на тёмном фото (результат — чёрный квадрат)



Рис. 4: Успешное выделение текста алгоритмом Оцу на том же тёмном фото

Случай 2: Пересвеченное изображение (блик). Яркость фона 250, текста 180. Базовый метод залил всё белым цветом (все пиксели > 127). Алгоритм Оцу снова успешно отделил бледно-серый текст от белого фона, переведя его в контрастный чёрный.

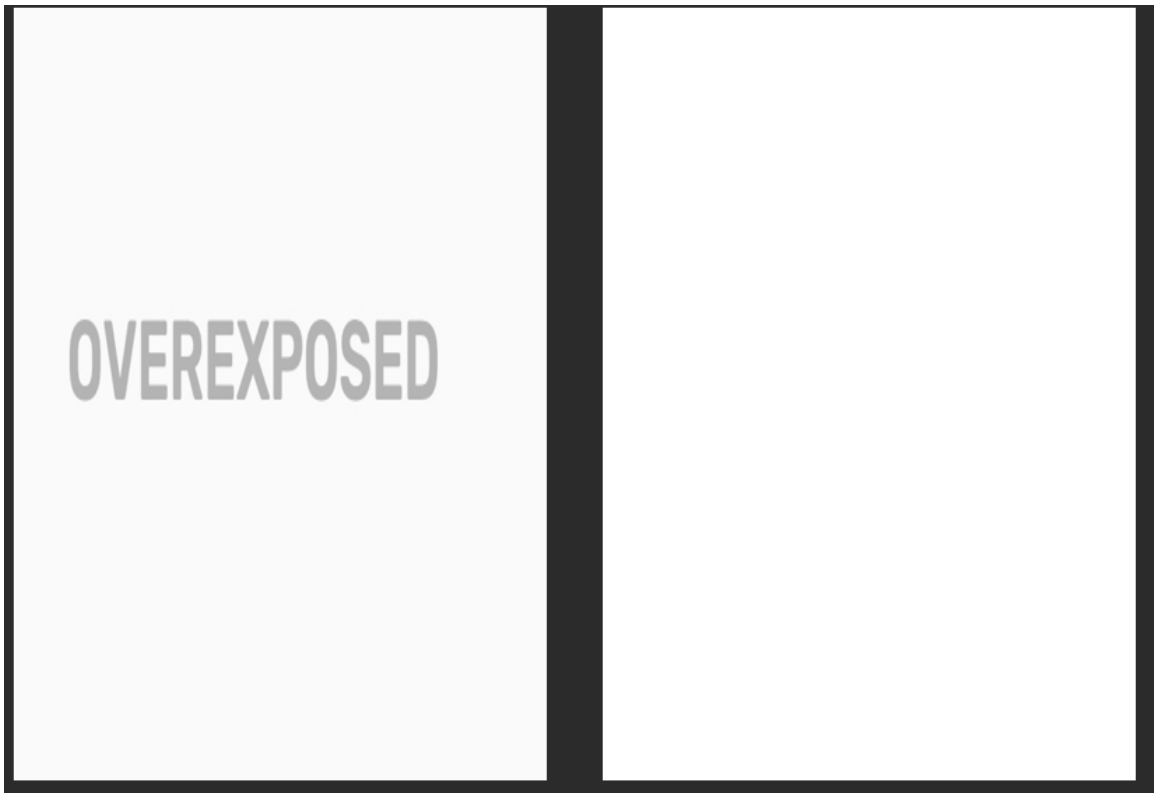


Рис. 5: Сбой базового метода на пересвеченном фото (результат — белый квадрат)



Рис. 6: Успешная обработка пересвеченного фото алгоритмом Оцу

6 Выводы

В ходе выполнения лабораторной работы я столкнулся с рядом концептуальных особенностей алгоритмов компьютерного зрения и пришёл к следующим выводам:

1. Ограниченность линейной математики и проблема шума.
Реализация пространственной свёртки показала, что фильтры не обладают семантическим пониманием картинки. Ядро Лапласа просто ищет любые математические перепады значений матрицы. Это приводит к серьёзной проблеме: алгоритм воспринимает полезные контуры (края объектов) и вредный цифровой шум (зернистость матрицы камеры) как одно и то же. Усиливая резкость объекта, мы неизбежно усиливаем шум. Из этого я усвоил важное правило построения конвейеров обработки: высокочастотным фильтрам всегда должно предшествовать низкочастотное сглаживание (например, размытие по Гауссу) для подавления мелких артефактов.

2. Хрупкость жёстко заданных параметров.
Тестирование базовой бинаризации с константой $T = 127$ наглядно продемонстрировало, почему такие методы неприменимы в production-системах. Алгоритм, отлично работающий на идеальном скане, моментально ломается, если на документ падает тень или свет от лампы (пересвет/недосвет). Я убедился, что алгоритмы, полагающиеся на абсолютные значения яркости пикселей, нежизнеспособны в реальном физическом мире с его неравномерным освещением.

3. Преимущество статистического абстрагирования.
Изучение алгоритма Оцу стало главным открытием работы. Вместо того чтобы в лоб сравнивать каждый пиксель, алгоритм переходит на уровень выше — анализирует распределение вероятностей (гистограмму). Он ищет не конкретный "чёрный"цвет,

а математическую пропасть (межклассовую дисперсию) между двумя любыми скоплениями пикселей. Оказалось, что для алгоритма Оцу нет разницы между "чёрным текстом на белом фоне"и "светло-серым текстом на белом фоне"— он одинаково успешно сегментирует объекты независимо от глобальной освещённости сцены.

4. Технические аспекты работы с матрицами.
При программной реализации я столкнулся с тем, что цветные изображения в OpenCV хранятся в инвертированном порядке байтов (BGR вместо RGB). Если напрямую передать такой массив NumPy в графический интерфейс (CustomTkinter/PIL), цвета искажаются (красный становится синим). Это научило меня тому, что в компьютерном зрении цифровое изображение — это просто многомерный тензор, и корректная конвертация пространств (`cv2.cvtColor`) является критически важным шагом перед любой отрисовкой.

7 Ссылки

GitHub репозиторий по компьютерной графике: `https://github.com/VadimO111/cg_bsu`.